Universidad de Buenos Aires
Facultad de Ciencias Exactas y Naturales
Departamento de Computación

[your title here]

Tesis de Licenciatura en Ciencias de la Computación

Rafael Romani

Director: Santiago Cifuentes
Codirector: Santiago Figueira
Buenos Aires, 2026

[YOUR TITLE HERE]

[Acá va el resumen]

Palabras claves: [acá van las palabras clave]

[YOUR TITLE HERE]

[write your abstract here]

Keywords: [write your keywords here]

[acá van los agradecimientos]

CONTENTS

1. INTRODUCTION

1.1 Uso y poder de los LLMS

In 2017, a group of researchers working at Google published a paper introducing a technology that revolutionized society. In *Attention is All You Need*, they presented the transformers, today's core of large language models (LLMs) and generative AI.

This architecture was build intended as text sequence-to-sequence model for machine translation but today has applications in various fields as natural language processing, computer vision or speech recognition.

The first model implementing transformers open for public use was ChatGPT by OpenAI. It made its debut in November 2022 and by December 2024 it reported 300 million global weekly users. Now there are many models competing for this market as Claude, from Anthropic, or Gemini from Google.

These models can be all be grouped under the generative AI tools. This is a technology that is arguably the most rapid technology adopted by society. In the second half of 2025, roughly one in six people worldwide were using generative AI tools. not sure how to cite this https://www.microsoft.com/en-us/research/wp-content/uploads/2026/01/Microsoft-AI-Diffusion-Report-2025-H2.pdf

LLMs are being used not only as chatbots for asking small questions, but are shaping the industry and future. For example, [?] found that more than 17% of the Computer Science's papers published between January 2020 and February 2024 on the *arXiv*, *bioRxiv*, and *Nature* portfolio journals had sentences heavily edited by ChatGPT.

SF: Lo citás como miscelaneous

Missing usage in industry

So naturally, the question about their computing power pops up. Are this kind of models really capable of doing the stuff for what they are being used?

- well-suited to parallelism enables transformer models to take full advantage of the power and speed offered by GPUs during both training and inference. This possibility, in turn, unlocked the opportunity to train transformer models on unprecedentedly massive datasets through self-supervised learning.
- chatgpt, claude, gemini,
- every day users
- number of users?
- use in the industry?
- other uses besides the chatbots?

cite this https://www.ibm.co model

1.2 Pero qué es un LLM?

1.3 Formalizaciones previas y resultados que ignoran los aspectos probabilísticos

1.4 Objetivos y contribuciones

2. THEORETICAL FRAMEWORK

2.1 Transformers

In this work we are going to analyze transformers as language recognizers. A transformer is a machine that works over an alphabet Σ. It has only one tape from where it reads the input and in an autoregressive

SF no sé bien a qué se refiere con autoregressive, pero tiene que quedar claro que solo imprime nuevos símbolos de a uno y que no puede modificar símbolos ya escritos. Quizá comparar con autómata vs TM

way writes in it more tokens. There are two distinguished tokens called **decision symbols**. Once the transformer writes one of those, it halts. We say that a transformer accepts a word if, when it halts, the decision symbol printed is the accepting one (q_{accept}). Analogously, a transformer rejects a word if the decision symbol printed is the rejecting one (q_{reject}).

In each step, the transformer maps the content of the tape to a probability distribution over Σ. Then, depending on this distribution, the transformer prints the next token. The transformer iterates this process until it halts. These iterations are called **chain of thought**.

RR: Agregar una cita

We split the process of generating the next token in two separated steps. First, the computation of the probability distribution over the alphabet which we call distribution generation.

SF así como hiciste con cot, definí nuevos términos con un estilo especial. Yo uso defstlye como comando y después decido si es itálica, negrita, etc

RR defino distribution generation y token selection más abajo y los puse en negrita. Debería marcarlos también acá?

Second, the selection of the token based on the probability distribution, which we call token by a process called token selection. In this work we analyze and compare two different algorithms for token selection.

In most of the literature, the transformers operate over an infinite tape. This is far from the real models, thus the transformers studied in this thesis work over a finite tape. The size of the tape is defined, for each input size n, as budget$(n) + n$. It is important to remark that the size of the tape bounds the number of steps of CoT that the transformer makes. For an input word of length n, no computation can take more than budget(n) steps because every step write a token in the tape.

SF: los pasos son del cómputo, nodel transformer. Revisar en todo el texto

SF mejorar esta última oración, no se entiende

RR ahí va mejor?

The process of distribution generation is dependent on many parameters. In the real models, these parameters are the ones calculated during the training.

SF Revisar casos como estos: These parameters, in the real models, are the ones calculated during the training. → In the real models, these parameters are the ones calculated during the training.

We refer to them as θ and consider them as part of the definition of a transformer.

We note $\mathsf{TF}(\alpha)$ to the token outputted by one step of the transformer TF when the tape content is $\alpha \in \Sigma^*$. The autoregressive token generation is defined recursively. The token outputted after i steps is $\mathsf{TF}^i(\alpha) := \mathsf{TF}^{i-1}(\alpha\mathsf{TF}(\alpha))$ with base case $\mathsf{TF}^1(\alpha) := \mathsf{TF}(\alpha)$. In other words, the output of the ith step is $\sigma_i = \mathsf{TF}^i(\alpha) = \mathsf{TF}(\alpha\sigma_1 \dots \sigma_{i-1})$ where $\sigma_j \in \Sigma$ for all j.

We note $\mathsf{TF}^*(\alpha)$ for the token written by the transformer when it halts, i.e., at the end of the computation, when the input is α. As said before, the transformer only halts when it prints a decision symbol (q_{accept} or q_{reject}). Therefore, $\mathsf{TF}^*(\alpha) \in \{q_{accept}, q_{reject}\}$.

As stated before, the **distribution generation** is a mapping of the content of the tape to a probability distribution over Σ. There are different algorithms for this process in the literature. All the transformers considered in this work use the one presented by [**?**]. This algorithm consists of four parts: a token embedding layer (TE), a position encoding layer (PE), an output linear layer (OUTPUT) and a stack of L identical decoder layers, where L is also called the depth of the model. Each decoder layer has two sub-layers: a multi-head self-attention layer (ATTN) and a position-wise fully-connected feed-forward network (FF). Each part and layer has its own trainable parameters. Then, we can define the distribution generation algorithm by its parameters $\theta = \{\theta_{\mathsf{TE}}, \theta_{\mathsf{PE}}, \{\theta^l_{\mathsf{ATTN}}, \theta^l_{\mathsf{FF}}\}_{l=0}^{L-1}, \theta_{\mathsf{OUTPUT}}\}$. The algorithm works over vectors of numbers of certain dimension d. The dimension is dependent on the size of the input word. We call $d(n)$ to the **embedding size** when the input word has size n.

SF: of reals?

RR: son $\mathbb{F}_{e,s}$. Puedo decir simplemente que son números por ahora?

Token Embedding: It is a mapping from Σ to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\mathsf{TE}} \in \mathbb{R}^{d\times|\Sigma|}$. For all $\sigma \in \Sigma$, we note $\theta_{\mathsf{TE}}(\sigma) \in \mathbb{R}^d$ to the mapping of token σ.

Position Encoding: It is a mapping from the positions of the tape to vectors in $\mathbb{R}^d$. This mapping is defined by $\theta_{\mathsf{PE}} \in \mathbb{R}^{d\times \mathrm{budget}(n)+n}$. For all position of the tape $1 \leq i \leq \mathrm{budget}(n) + n$ we note $\theta_{\mathsf{PE}}(i) \in \mathbb{R}^d$ to the mapping of position i.

SC Intuición de todas estas capas?

RR Lo hago en la intro? Me parece el mejor lugar para hacerlo

Self Attention Layer: It is a mapping from $(\mathbb{R}^d)^a$ to $(\mathbb{R}^d)^a$. Given parameters $\theta_{\mathsf{ATTN}} = (W_Q, W_K, W_V, W_O) \in \mathbb{R}^{d\times d} \times \mathbb{R}^{d\times d} \times \mathbb{R}^{d\times d} \times \mathbb{R}^{d\times d}$, the layer is defined by the algorithm 1. This layer is defined only as a single head attention layer because we can simulate 1 multi-head attention layer with any constantly many heads with multiple single-head attention layers[**?**].

SF: ¿qué es a?

RR: es la cantidad de elementos que hay en la cinta en ese paso. Hay un vector por cada elemento. Cada paso de ATTN consume todos los vectores y devuelve la misma cantidad de vectores. En el algoritmo aparece a. No se me ocurrió otra forma para notarlo.

Algorithm 1 Causal Self-Attention, ATTN

Input: Parameter $\theta_{\mathsf{ATTN}} = (W_Q, W_K, W_V, W_O)$ and embedding $h = (h_1, \dots, h_a) \in \mathbb{R}^d \times \dots \times \mathbb{R}^d$.
Output: Embedding $h' = (h'_1, \dots, h'_a) := \mathsf{ATTN}_{\theta_{\mathsf{ATTN}}}(h_1, \dots, h_a)$.
1: $q_i := W_Q h_i,\ k_i := W_K h_i,\ v_i := W_V h_i,\ \forall i \in [1, \dots, a]$
2: $s_i := \mathrm{softmax}(\langle q_i, k_1\rangle, \dots, \langle q_i, k_i\rangle) \| (0, \dots, 0)$
3: $h'_i := W_O \sum_{j=1}^a (s_i)_j v_j$

Feed Forward Network: It is a mapping from $\mathbb{R}^d$ to $\mathbb{R}^d$. Given $\theta_{\mathsf{FF}} = (W_1, b_1, W_2, b_2) \in \mathbb{R}^{d\times d} \times \mathbb{R}^d \times \mathbb{R}^{d\times d} \times \mathbb{R}^d$, the layer is defined as $\mathsf{FF}_{\theta_{\mathsf{FF}}}(h) := W_2\ \mathrm{relu}(W_1 h + b1) + b2$, where $\mathrm{relu} : \mathbb{R}^d \to \mathbb{R}^d$ replaces each position x_i by $\max(0, x_i)$.

SF Too many it's it's... Cambié algunos. Mejorar

Output Layer: It is a mapping from $\mathbb{R}^d$ to a probability distribution over Σ. Given $\theta_{\mathsf{OUTPUT}} \in \mathbb{R}^{|\Sigma|\times d}$, we define it as $\mathsf{OUTPUT}_{\theta_{\mathsf{OUTPUT}}}(h) := \mathrm{softmax}(\theta_{\mathsf{OUTPUT}} h)$.

All these parts come together for the distribution generator process in the algorithm 2.

Algorithm 2 Distribution generator

Input: Transformer parameter $\theta = (\theta_{\mathsf{TE}}, \theta_{\mathsf{PE}}, \{\theta^l_{\mathsf{ATTN}}, \theta^l_{\mathsf{FF}}\}_{l=0}^{L-1}, \theta_{\mathsf{OUTPUT}})$ and tokens of the tape $\alpha \in \Sigma^*$.

Output: distribution $p_\theta(\cdot|\alpha)$.

```
1: h_i^{(0)} ← θ_TE(α_i) + θ_PE(i), ∀1 ≤ i ≤ |α|
2: for l = 0, ..., L − 1 do
3:     (h_1^{(l+0.5)}, ..., h_{|α|}^{(l+0.5)}) ← (h_1^{(l)}, ..., h_{|α|}^{(l)}) + ATTN_{θ_ATTN^{(l)}}(h_1^{(l)}, ..., h_{|α|}^{(l)})
4:     h_i^{(l+1)} ← h_i^{(l+0.5)} + FF_{θ_FF^{(l)}}(h_i^{(l+0.5)}), ∀1 ≤ i ≤ |α|
5: end for
6: p_θ(·|α) ← OUTPUT_{θ_OUTPUT}(h_{|α|}^{(L)})
```

The OUTPUT layer and the ATTN layer use a softmax function $\mathbb{R}^n$ to $\mathbb{R}^n$ defined as $\text{softmax}(x)_i = exp(x_i)/\sum_{j=1}^n exp(x_j)$.

SF Ejemplo de cómo mejorar la presentación para repetir menos y ser más claro:
The OUTPUT layer and the ATTN layer uses a softmax function. It is defined from $\mathbb{R}^n$ to $\mathbb{R}^n$ for all n as $\text{softmax}(x)_i = exp(x_i)/\sum_{j=1}^n exp(x_j)$.
$\rightarrow$
The OUTPUT layer and the ATTN layer use a function $\mathbb{R}^n \rightarrow \mathbb{R}^n$ defined by $\text{softmax}(x)_i = exp(x_i)/\sum_{j=1}^n exp(x_j)$.

Independientemente de la presentación, no sé si queda claro el tema del n ("for all n"). ¿Es una familia de funciones softmax_n?

RR Creo que formalmente es una familia de funciones porque es distinta para cada tamaño de vector, pero en el fondo hace siempre lo mismo. Quizás borrando el para todo n queda más claro? No creo que valga la pena introducir que es una familia.

The second component of a transformer is the **token selector**. This algorithm consumes the probability distribution computed by the distribution generator and returns a token.

In this work we study two selectors, we call them **deterministic selector** and **probabilistic selector**. Let $p : \Sigma \to [0, 1]$ be the output of the distribution generator. Then, the deterministic selector always returns the token with the largest probability (argmax over p). On the other hand, the probabilistic selector works non-deterministically. It returns a token $\sigma \in \Sigma$ with probability $p(\sigma)$.

SF: Σ^*?

RR: No, el token selector consume la distribución que dado un token devuelve su probabilidad

RR Add an image representing the transformer architecture

Definition 2.1.1 (Deterministic transformer)**.** We call a transformer deterministic when the token selector is the deterministic selector. We say that a family of deterministic transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ recognizes a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \mathsf{TF}^*_{|\alpha|}(\alpha) = q_{accept}$$

As mentioned in chapter 1, there is a theoretical gap in the area. A study of non-determinism in transformers is missing.

RR: está bien dicho theoretical gap?

SF: no, theoretical gap no me suena bien acá. ¿A qué te referís?

When TF is a deterministic transformer, $\mathsf{TF}^i(\alpha)$ is an exact symbol for all $\alpha \in \Sigma^*$ and $i \in \mathbb{N}$. On the contrary, when the token selection process of the transformer samples the distribution, it becomes a random variable. Therefore, we introduce the following definition.

Definition 2.1.2 (Probabilistic transformer)**.** We call probabilistic transformer to the transformers that use the probabilistic selector. We say that a family of probabilistic transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ recognizes a language $\mathcal{L}$ when:

$$\alpha \in \mathcal{L} \iff \Pr[\mathsf{TF}^*_{|\alpha|}(\alpha) = q_{accept}] > 2/3$$

SF ¿Dónde quedó el budget en estas definiciones?

RR El budget solo se usa para acotar la cantidad de pasos de CoT. También aparece en el position embeding para poder argumentar que es una función que hace cualquier cosa pero con dominio finito. Si el dominio no fuera finito se rompe la demo de que podes simular un paso de cómputo con un circuito.

2.2 Finite precision modeling

I'm missing good symbols for ∞ y $-\infty$.

In practice, training and inference of transformers are typically done with 16- or 32-bit floating point numbers and there is active work in reducing that number. That's why in this thesis we follow the decision of [?] to work on constant-precision transformers, where the output of each arithmetic operation is rounded to the closest floating point number representable by a fixed number of digits following IEEE 754 standard. This analysis avoids the unrealistic infinite precision assumption made in prior works [?]. The definitions stated in the section are inherited from [?].

SF: gramática

SF The definitions stated in this section are taken from [?].

Below we give a formal definition of the floating-point number and rounding operation. We use $\phi(a) = \sum_{i=1}^{k} 2^{k-i} a_i$ to denote the value of the binary number represented by $a \in \{0,1\}^k$ for any $k \in \mathbb{N}^+$.

SF Antes de la definición, explicar cuál es la representación que se usa, qué es el exponent y el significand. ¿Es base y exponente? Y de sing... Poner un ejemplito

Definition 2.2.1 (Floating-point Representation)**.** Let e be the number of bits for exponents and s be the number of bits for significand. A $(e + 2s + 1)$-bit binary string $a = (a_1, a_2, \ldots a_{e+2s+1}) \in \{0,1\}^{e+2s+1}$ is a floating-point binary representation of number $\phi_{e,s}(a) \triangleq \text{sign}(a) \cdot 2^{\text{exponent}(a)} \cdot \text{significand}(a)$ with e-bit exponent and $2s$-precision, where the sign is $\text{sign}(a) \triangleq 2a_1 - 1$, the significand is $\text{significand}(a) \triangleq 2^{-s}\phi(a_{2:2s+1})$, and the exponent is $\text{exponent}(a) \triangleq \phi(a_{2s+2:2s+e+1}) - 2^{\max(0,e-1)}$. We define the set of numbers expressible with e bits of exponent and s bits of significand as $\mathbb{F}_{e,s}$. We define $\infty_{e,s}$ as the maximum number in $\mathbb{F}_{e,s}$. Similarly, we define $-\infty_{e,s}$ as the minimum. Since in this work we only analyze constant precision transformers, we will omit the subscripts of ∞ and $-\infty$.

Definition 2.2.2 (Correct Rounding)**.** For any $x \in \mathbb{R}$, we define correct rounding $\text{round}(x, \mathbb{F}_{e,s})$ as the closest number to x in $\mathbb{F}_{e,s}$. We break the tie by picking the one with a smaller absolute value. In particular, we denote the rounding operation with e-bit exponent, $2s$-bit precision by $\text{round}_{e,s}(\cdot) \triangleq \text{round}(\cdot, \mathbb{F}_{e,s})$, which is also denoted by $[\cdot]_{e,s}$ for convenience. We extend the definition of round and $\text{round}_{e,s}$ to vector inputs by rounding coordinate-wisely.

Our notion of floating-point number simplifies the IEEE 754 Standard for Floating-point Arithmetic [?] by removing ∞ and $-\infty$. When overflow happens, we always round the output to ∞, and when underflow happens to $-\infty$. For unary functions like $\exp(\cdot)$ and binary functions including addition, subtraction, multiplication, and division, we simply define their rounded version by rounding their outputs. Whenever division by 0 happens, we define the output as 0.[1]

Next, we define finite-precision summation over more two numbers by decomposing it as a chain of rounded binary addition in a fixed order. [2]

SF: gramática

Definition 2.2.3 (Summation with Iterative Rounding)**.** For any $s, n \in \mathbb{N}^+$ and vector $x \in \mathbb{R}^n$, we define summation with iterative rounding to e bit exponent and $2s$-bit precision as $\mathsf{sum}_{e,s} : \cup_{n\in\mathbb{N}^+} (\mathbb{F}_{e,s})^n \to \mathbb{F}_{e,s}$, where for any $n \in \mathbb{N}^+$ and $x \in \mathbb{R}^n$,

$$\mathsf{sum}_{e,s}(x) \triangleq \left[\left[\left[[x_1 + x_2]_{e,s} + x_3\right]_{e,s} + \cdots x_{n-1}\right]_{e,s} + x_n\right]_{e,s} .$$

We further define the following operations:

- Finite-precision inner product: $\langle x, y\rangle_{e,s} \triangleq \mathsf{sum}_{e,s}(x \odot y)$;
- Finite-precision matrix product: $(A \times_{e,s} B)_{i,j} \triangleq \left\langle (A_{i,:})^\top, B_{:,j} \right\rangle_{e,s}$;
- Finite-precision softmax: $\mathrm{softmax}_{e,s}(x) \triangleq \left[[\exp(x)]_{e,s} / \mathsf{sum}_{e,s}([\exp(x)]_{e,s})\right]_{e,s}$.

From the saturation and the iterative rounding presented by [?], some properties appear that are important to remark:

- $exp(-\infty) = 0$
- $exp(\infty) = \infty$
- $-\infty \cdot \infty = -\infty$
- $\infty \cdot \infty = \infty$
- $\forall x \in \mathbb{F}_{e,s}$ it holds that $x + 0 = x$
- $\forall x \in \mathbb{F}_{e,s}$ it holds that $x - x = 0$
- $\forall x \in \mathbb{F}_{e,s}$ it holds that $x + \infty + \infty + -\infty = 0$

2.3 Transformer Complexity classes

Maybe rethink this title?

As discussed in the first chapter, there are different resources to bound in the transformers to limit their expressive power. In this thesis, we analyze the effect of bounding

[1] Notice that the only division is in the softmax. If it's the one in the attention mechanism, it's reasonable to give an attention score of 0 to every vector. If it's the one in the output layer it means that all symbols have an extremely small probability, therefore is reasonable to assign the same probability to all of them.

[2] Technically speaking, instead of a chain, the summation could also proceed like a tree.

the number of steps in the chain of thought that the transformers are allowed to make before answering. With this in mind, we define the following complexity classes.

Given two functions $T(n)$ and $d(n)$ we define the complexity class $\mathsf{CoT}[T(n), d(n)]$. Informally, a language $\mathcal{L}$ is in $\mathsf{CoT}[T(n), d(n)]$ if and only if there is a deterministic family of transformers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it.

Definition 2.3.1 ($\mathsf{CoT}[T(n), d(n)]$)**.** A language $\mathcal{L}$ is in $\mathsf{CoT}[T(n), d(n)]$ if and only if there is an integer L and two functions $T'(n) = O(T(n)), d'(n) = O(d(n))$, such that for every positive integer n, there is an L-layer deterministic transformer, denoted by TF_n with embedding size $d'(n)$, that outputs $\mathcal{L}(\alpha)$ given any input α in Σ^n, using $T'(n)$ steps of chain of thought.

SF Explicar qué es $\mathcal{L}(\alpha)$.

RR No es notación estándar?

In the same way, we define the classes of languages recognized by probabilistic transformers. Given two functions $T(n)$ and $d(n)$ we define the complexity class $\mathsf{RCoT}[T(n), d(n)]$. Informally, a language $\mathcal{L}$ is in $\mathsf{RCoT}[T(n), d(n)]$ if and only if there is a probabilistic family of transformers with constant depth, embedding size $d(n)$ and budget $T(n)$ that recognizes it.

Definition 2.3.2 ($\mathsf{RCoT}[T(n), d(n)]$)**.** A language $\mathcal{L}$ is in $\mathsf{RCoT}[T(n), d(n)]$ if and only if there is an integer L and two functions $T'(n) = O(T(n)), d'(n) = O(d(n))$, such that for every positive integer n, there is an L-layer probabilistic transformer, denoted by TF_n with embedding size $d'(n)$, that output $\mathcal{L}(\alpha)$ given any input α in Σ^n, using $T'(n)$ steps in the chain of thought with probability higher than $2/3$.

2.4 Circuit Complexity

2.5 Known results

3. GENERAL PROPERTIES

In this chapter we study the properties of $\mathsf{CoT}[T(n), d(n)]$ classes and the primitive operations that transformers can perform. We define the notion of normalized transformer and prove that $\mathsf{CoT}[T(n), d(n)]$ classes are closed under boolean operations.

SF: mejorar la redacción

RR: ahí va mejor?

3.1 Transformers Primitives

SF En esta sección se presentan las primitivas que se van a usar a lo largo de la tesis. Estas primitivas son operaciones que pueden ser implementadas por un transformer con ciertas características.... bla bla. Motivar y explicar por qué se necesita esta sección. Mencionar cuáles van a ser las primitivas y decir que la explicación de cada una de ellas se encuentra en las subsecciones correspondientes.

3.1.1 Flagging Tokens and Positions

For complex operations inside the loop of the transformer we will need to identify when a vector comes from a certain token or position. In this section we present how to mark the vectors with flags during the embedding.

For flagging a token we use the token embedding, and in the same way, for flagging a position we use the position encoding. Either way, the strategy is exactly the same. The embedding dimension is extended to allocate the flags. A new coordinate is added per flag. Therefore, using a constant number of flags only adds a constant to the embedding dimension. Thus, if the flagging is made in a family of transformers its $\mathsf{CoT}[T(n), d(n)]$ class does not change.

For example, let v_σ and v_τ be vectors of a transformer TF where v_σ comes from embedding the token σ and v_τ from the token τ (with $\sigma \neq \tau$). We can define a transformer TF' that behaves like TF but recognize when a vector comes from embedding the token σ. TF' have embedding dimension $d' = d + 1$ where d is the embedding of TF.

$$v_\sigma = \begin{pmatrix} x \\ y \\ z \end{pmatrix} \to v'_\sigma = \begin{pmatrix} x \\ y \\ z \\ f_{\mathsf{on}} \end{pmatrix} \qquad\qquad v_\tau = \begin{pmatrix} i \\ j \\ k \end{pmatrix} \to v'_\tau = \begin{pmatrix} i \\ j \\ k \\ f_{\mathsf{off}} \end{pmatrix}$$

Fig. 3.1: Change in a flagged and unflagged vector

Formally, for flagging the token σ we define the token embedding function of TF' : $\Sigma \to \mathbb{R}^{d+1}$ as

$$(\theta'_{TE}(\tau))_j = \begin{cases} (\theta_{TE}(\tau))_j & \text{if } j \neq d+1 \\ f_{\mathsf{on}} & \text{if } j = d+1 \text{ and } \tau = \sigma \\ f_{\mathsf{off}} & \text{if } j = d+1 \text{ and } \tau \neq \sigma \end{cases}$$

where f_{on} is the value that marks the vector as flagged and f_{off} as unflagged. Those values can be defined arbitrary for each flag. In this case, the coordinate inserted for the flag is

RR: arbitrary no es la palabra que quiero me parece the last one, but it can be added anywhere.

The other components of TF' ignore $d+1$ the coordinate. The position encoding of TF' its equal to the position encoding of TF, except it puts a 0 in the $d+1$ coordinate (not present in TF) independently of the position. The matrix of the ATTN, FF and OUTPUT layers of TF' are the ones of TF extended with 0 in the $d+1$ rows and columns to match the change in embedding dimension.

TF' behaves exactly as TF except it has one more coordinate in the embedding. Said coordinate represent the flag and in every vector has only values v_{on} or v_{off}.

This primitive works also as a mechanism to extend the embedding dimension without changing the behavior. If v_{on} and v_{off} are defined as 0, the resulting transformer is the same as the initial with an extra coordinate that it is always 0.

3.1.2 Preserve and ignore coordinates through ATTN and FF

When defining new transformers over existing ones we will need to extend the embedding dimension to store new information. For preserving the behavior of the original transformers through the attention and feed forward layers, we need to build a mechanism that ignores the extra coordinates. For example, this mechanism is used for preserving flags during ATTN and FF layers that don't use them.

Let TF be a transformer of embedding dimension d and assume we are constructing a transformer TF' of embedding dimension $d' = d+1$. Let W_Q, W_K, W_V, W_O be the matrix of an ATTN layer of TF and assume we want to perform the same layer in TF' but simulating that the ith coordinate does not exist. Notice that the embedding dimension of TF' has one more coordinate than TF's. Defining the matrix of the layer W'_Q, W'_K, W'_V, W'_O as W_Q, W_K, W_V, W_O but inserting a row and column of zeros in the ith positions gives the desired ATTN layer. Note that because the ith row of W'_O is zero, the output vectors of the ATTN layer are 0 in the ith coordinate. This is the point that makes it possible to not just ignore the value of the ith coordinate, but to not affect it.

For preserving the coordinate but not ignore it, it is enough to have zeros in only its corresponding row of W'_O. This construction works for more than one coordinate. For extending it to a set of coordinates, it is only needed to make more rows and columns 0.

The same applies in the feed forward layer, making the ith column and row of the matrix and vectors of the layer zero, makes it ignore the value of the ith coordinate. Also in the result of the FF will appear a zero in the ith coordinate, thus also preserving the value.

SC diagrama?

RR how? pongo las matrices? Siento que a veces queda muy cargado. Quizás ahora se entiende mejor?

3.1.3 Add vector and constant functions

In this section we develop a mechanism for adding a fixed vector to a previously flagged one. This technique works even when there are more than one flagged vector.

Assume there is a flag with values $f_{\mathsf{on}} = 1$ and $f_{\mathsf{off}} = 0$. For clarity, suppose the flag is in the last coordinate. Let v be the vector we want to add. We define a mechanism that for all h such that $h_d = f_{\mathsf{on}}$, it replaces them with $h+v$.

The operation is performed with only one FF layer. We construct a layer such that $FF(h) = \mathbb{I}(h_d = f_{\mathsf{on}}) \cdot v$. Said layer is achieved with the following parameters.

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & \dots & 0 & v_1 \\ \vdots & \ddots & \vdots & \vdots \\ 0 & \dots & 0 & v_d \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ \vdots \\ 0 \end{pmatrix}$$

Notice that if the coordinate for the flag had been the kth instead of the last one, then v would have appeared in the kth column of W_2 instead of the last.

This technique is that it can be used to apply a constant function to the flagged vectors. This is achieved exploiting the finite representation system. We can saturate the vector to the ∞ and then make it 0 to finally add our objective. This holds since for all $x, c \in \mathbb{F}_{e,s}$ it happens that $x + \infty + \infty + -\infty + c = c$. Because as mentioned in section 2.2, $x + \infty + \infty = \infty$ and $\infty + -\infty = 0$.

The constant function $f(h) = v$ is performed adding four vectors such that performs this trick. First, it is necessary to add twice the vector with only ∞, then add a vector with only $-\infty$ and finally add v. As is visually presented in equation 3.1.3.

$$h \leftarrow h + \begin{pmatrix} \infty \\ \vdots \\ \infty \end{pmatrix} + \begin{pmatrix} \infty \\ \vdots \\ \infty \end{pmatrix} + \begin{pmatrix} -\infty \\ \vdots \\ -\infty \end{pmatrix} + v$$

Fig. 3.2: Constant function

RR más arriba digo ecuación pero latex dice que es una figura. Tampoco estoy seguro qué caption ponerle. Quizás es mejor si directamente no la referencio?

3.1.4 Make a vector ignore others in an ATTN layer

We show how to make a vector ignore others in the ATTN layers, which will be useful for later constructions. A vector h_i ignores vector h_j when the attention score is 0 which can be achieved making $\langle q_i, k_j \rangle = -\infty$.

We extend the embedding dimension with three more coordinates for preserving the attention score of the other vectors (modulo softmax).

RR no sé cómo decir esto sin dar muchas vueltas. Porque si ignoro un vector, naturalmente el attention score de los demás sube por la "normalización" que hace el softmax

We construct TF' over TF such that the lth vector ignores a set of vectors M in an ATTN layer, we are looking for said layer to respect:

$$\langle q'_i, k'_j \rangle = \begin{cases} -\infty & \text{if } i = l \wedge j \in M \\ \langle q_i, k_j \rangle & \text{if } i \neq l \vee j \notin M \end{cases}$$

This can be achieved saturating the finite representation since $\langle q'_l, k'_i \rangle = \sum_{x=1}^{d+3} (q'_l)_x (k'_i)_x$. If the last two terms of the summation are $-\infty$, then because of the iterative rounding, $\langle q'_l, k'_i \rangle = -\infty$. Since we don't want to change the attention scores of vectors not in M, when $i \notin M$ the last three terms should be 0.

We force $(k'_i)_{d+2}$ and $(k'_i)_{d+3}$ to be $-\infty$ when $i \in M$, and $(q'_i)_{d+2}$ and $(q'_i)_{d+3}$ to be 1 when $i = l$ and 0 otherwise. We also make $(k'_i)_{d+1}$ and $(q'_i)_{d+1}$ to be 0 for all i so the third to last term of the inner product is always 0. With these values, the only attention scores modified are the ones of the lth vector since every other will have 0 in the last three coordinates of the query thus making the last three terms of the inner product 0 and therefore not affecting the score.

This can be achieved with the following flags:

$$(h_i)_{d+1} = \begin{cases} 1 & \text{if } i = l \\ 0 & \text{otherwise} \end{cases} \qquad (h_i)_{d+2} = (h_i)_{d+3} = \begin{cases} 1 & \text{if } i \in M \\ 0 & \text{otherwise} \end{cases}$$

Now the query and key matrix are extended as:

$$W'_Q = \begin{pmatrix} W_Q & 0^{\mathbb{R}^{d\times 1}} & 0^{\mathbb{R}^{d\times 1}} & 0^{\mathbb{R}^{d\times 1}} \\ 0^{\mathbb{R}^{d\times 1}} & 0^{\mathbb{R}^{1\times 1}} & 0^{\mathbb{R}^{1\times 1}} & 0^{\mathbb{R}^{1\times 1}} \\ 0^{\mathbb{R}^{d\times 1}} & 1^{\mathbb{R}^{1\times 1}} & 0^{\mathbb{R}^{1\times 1}} & 0^{\mathbb{R}^{1\times 1}} \\ 0^{\mathbb{R}^{d\times 1}} & 1^{\mathbb{R}^{1\times 1}} & 0^{\mathbb{R}^{1\times 1}} & 0^{\mathbb{R}^{1\times 1}} \end{pmatrix} \quad W'_K = \begin{pmatrix} W_K & 0^{\mathbb{R}^{d\times 1}} & 0^{\mathbb{R}^{d\times 1}} & 0^{\mathbb{R}^{d\times 1}} \\ 0^{\mathbb{R}^{d\times 1}} & 0^{\mathbb{R}^{1\times 1}} & 0^{\mathbb{R}^{1\times 1}} & 0^{\mathbb{R}^{1\times 1}} \\ 0^{\mathbb{R}^{d\times 1}} & 0^{\mathbb{R}^{1\times 1}} & -\infty^{\mathbb{R}^{1\times 1}} & 0^{\mathbb{R}^{1\times 1}} \\ 0^{\mathbb{R}^{d\times 1}} & 0^{\mathbb{R}^{1\times 1}} & 0^{\mathbb{R}^{1\times 1}} & -\infty^{\mathbb{R}^{1\times 1}} \end{pmatrix}$$

Then, $\langle q'_i, k'_j \rangle = -\infty$ if $i = l$ and $j \in M$, but $\langle q'_i, k'_j \rangle = \langle q_i, k_j \rangle$ otherwise.

RR: Como referencio que eso vale por las cuentas que tengo acá abajo?

$$\begin{aligned} \langle q'_i, k'_j \rangle &= \sum_{x=1}^{d+3} (q'_l)_x (k'_i)_x = \langle q_i, k_j \rangle + (q'_l)_{d+1}(k'_i)_{d+1} + (q'_l)_{d+2}(k'_i)_{d+2} + (q'_l)_{d+3}(k'_i)_{d+3} \\ &= \langle q_i, k_j \rangle + 0 + (q'_l)_{d+2}(k'_i)_{d+2} + (q'_l)_{d+3}(k'_i)_{d+3} \\ &= \langle q_i, k_j \rangle (q'_l)_{d+2}(k'_i)_{d+2} + (q'_l)_{d+3}(k'_i)_{d+3} \\ &= \begin{cases} \langle q_i, k_j \rangle + 1 \cdot -\infty + 1 \cdot -\infty & \text{if } i = l \wedge j \in M \\ \langle q_i, k_j \rangle + 1 \cdot 0 + 1 \cdot 0 & \text{if } i = l \wedge j \notin M \\ \langle q_i, k_j \rangle + 0 \cdot -\infty + 0 \cdot -\infty & \text{if } i \neq l \wedge j \in M \\ \langle q_i, k_j \rangle + 0 \cdot 0 + 0 \cdot 0 & \text{if } i \neq l \wedge j \notin M \end{cases} \\ &= \begin{cases} \langle q_i, k_j \rangle + -\infty + -\infty & \text{if } i = l \wedge j \in M \\ \langle q_i, k_j \rangle + 0 + 0 & \text{if } i = l \wedge j \notin M \\ \langle q_i, k_j \rangle + 0 + 0 & \text{if } i \neq l \wedge j \in M \\ \langle q_i, k_j \rangle + 0 + 0 & \text{if } i \neq l \wedge j \notin M \end{cases} \\ &= \begin{cases} -\infty & \text{if } i = l \wedge j \in M \\ \langle q_i, k_j \rangle & \text{otherwise} \end{cases} \end{aligned}$$

3.1.5 Linear transformations

Although it looks that transformers are a model that should be able to perform linear transformations in a native way, it is not the case. Applying a linear transformation to a vector is not direct since in every step of ATTN and FF the result of the layer is added to the original vector instead of replacing it. An intuitive idea would be to apply the transformation $M - I$ when we want to transform the vector x to Mx, since

$x + (M - I)x = x + Mx - Ix = Mx$. This does not hold in our model since the addition and multiplication is not commutative in $\mathbb{F}_{e,s}$. For avoiding the representation error, we will extend the embedding dimension to get more memory per vector such that we can store in new coordinates the value of Mx and then replace it in the original ones.

Let f be the number of a vector. We show a mechanism to replace it with the result of applying the linear transformation M to it. We perform the operation with two consecutive ATTN layers where h_f only attends to itself. Therefore, after these two layers h_f will be replaced with Mh_f.

RR: uso f porque viene de flagged

We need to double the embedding size from d to $2d$ (plus some flags) since we need to store the result of the transformation in some coordinates (the added $d+1$ to $2d$) as a middle step. In the first ATTN layer, we compute the transformation saving its result in the coordinates $d+1$ to $2d$. Then, in the second layer we move the result to the right place.

$$\widetilde{h_f} = \begin{pmatrix} (h_f)_1 \\ \vdots \\ (h_f)_d \\ 0 \\ \vdots \\ 0 \end{pmatrix} \xrightarrow{\text{first ATTN layer}} \widetilde{h_f} = \begin{pmatrix} 0 \\ \vdots \\ 0 \\ (Mh_f)_1 \\ \vdots \\ (Mh_f)_d \end{pmatrix} \xrightarrow{\text{second ATTN layer}} \widetilde{h_f} = \begin{pmatrix} (Mh_f)_1 \\ \vdots \\ (Mh_f)_d \\ 0 \\ \vdots \\ 0 \end{pmatrix}$$

We assume that the $d+1$ to $2d$ coordinates (used as memory) are set to 0 by the layers previous to this mechanism. This can be achieved maintaining them as 0 since the beginning or transformed to 0 with the primitive described in the section 3.1.3.

Let us construct each of the ATTN layers. As said before, in both layers we will need for $\widetilde{h_f}$ to only attend to itself. Therefore, we define W_Q ans W_K using the strategy of the section 3.1.4.

Recall from the definition of the ATTN mechanism (1) that the output of the ATTN layer is $h'_i = W_O \sum_{j=1}^{a} (s_i)_j v_j$. By this point, h'_f is looking like

$$h'_f = W_O \sum_{j=1}^{a} (s_f)_j v_j = W_O v_f = W_O (W_V \widetilde{h_f})$$

Then, defining W_O and W_V as follows

SF: dividir claramente en primera y segunda layer. Podés usar itemize o paragrph.

$$W_V = id \qquad\qquad W_O = \begin{pmatrix} -id^{\mathbb{R}^{d\times d}} & 0^{\mathbb{R}^{d\times d}} \\ M & 0^{\mathbb{R}^{d\times d}} \end{pmatrix}$$

makes us get $h'_f = (-(h_f)_1, \ldots, -(h_f)_d, (Mh_f)_1, \ldots, (Mh_f)_d)$, which added to $\widetilde{h_f}$ gives us what we were looking for after the first layer.

RR: estoy de acuerdo, pero no estoy seguro cómo hacerlo

For the second ATTN layer, we use the same mechanism for making $\widetilde{h_f}$ only attend to itself. And defining W_V and W_O as follows

$$W_V = id \qquad\qquad W_O = \begin{pmatrix} 0^{\mathbb{R}^{d\times d}} & id^{\mathbb{R}^{d\times d}} \\ 0^{\mathbb{R}^{d\times d}} & -id^{\mathbb{R}^{d\times d}} \end{pmatrix}$$

we get $h'_f = ((Mh_f)_1, \ldots, -(Mh_f)_d, -(Mh_f)_1, \ldots, -(Mh_f)_d)$, which added to $\widetilde{h_f}$ gives us what we were looking for after the second layer.

Notice that after these two layers, the values of h_i for all $i \neq f$ had been modified in ways not studied in this work.

RR no sé cómo decir esto. El comentario anterior de que habían sido corrupted hacía referencia a esto.

3.1.6 Max coordinate

RR Si a = b mi construcción pone 0 en ambas. Esto lo uso para normalizar. En el paper original no dicen cómo resolver los empates del argmax. Puedo poner un footnote de que en ese caso la construction nuestra pone dos ceros pero no sé si es worth

We show how to take the maximum of a set of coordinates. This operation will be needed for later constructions.

We give a construction for taking the maximum of two coordinates, but it can be extended to a larger set repeating the operation as a tournament.

$$\begin{pmatrix} a \\ b \\ \vdots \end{pmatrix} \to \begin{cases} \begin{pmatrix} a \\ 0 \\ \vdots \end{pmatrix} & \text{if } a > b \\ \begin{pmatrix} 0 \\ b \\ \vdots \end{pmatrix} & \text{if } b > a \end{cases}$$

The maximum of two coordinates can be computed with the following primitives:

$$\begin{pmatrix} a \\ b \\ 0 \\ 0 \\ \vdots \end{pmatrix} \xrightarrow{\text{FF layer}} \begin{pmatrix} a \\ b \\ \text{relu}(a-b) \\ \text{relu}(b-a) \\ \vdots \end{pmatrix} \xrightarrow[\text{the coordinates with the relu}]{\text{Multiply twice by } \infty} \begin{pmatrix} a \\ b \\ \infty \cdot \mathbb{I}(a > b) \\ \infty \cdot \mathbb{I}(b > a) \\ \vdots \end{pmatrix} \longrightarrow$$

$$\xrightarrow[\substack{\text{coordinates with} \\ \text{the comparison}}]{\text{Divide by } \infty \text{ the}} \begin{pmatrix} a \\ b \\ \mathbb{I}(a > b) \\ \mathbb{I}(b > a) \\ \vdots \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a \cdot \mathbb{I}(a > b) \\ b \cdot \mathbb{I}(b > a) \\ 0 \\ 0 \\ \vdots \end{pmatrix} = \begin{cases} \begin{pmatrix} a \\ 0 \\ \vdots \end{pmatrix} & \text{if } a > b \\ \begin{pmatrix} 0 \\ b \\ \vdots \end{pmatrix} & \text{if } b > a \end{cases}$$

Implementation of the special operation

This operation is not a linear transformation, therefore we need to define a new technique. Notice that one of $\mathbb{I}_a$ and $\mathbb{I}_b$ is a 1 and the other is a 0. Then, $\mathbb{I}_b = 1 - \mathbb{I}_a$ and $\mathbb{I}_a = 1 - \mathbb{I}_b$.

$$\begin{pmatrix} a \\ b \\ \mathbb{I}_a \\ \mathbb{I}_b \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a \cdot \mathbb{I}_a \\ b \cdot \mathbb{I}_b \\ 0 \\ 0 \end{pmatrix}$$

This operation is performed with three layers of FF. Naturally, after each layer a new vector is added to the original. The vectors we add are the following:

$$\begin{pmatrix} a \\ b \\ \mathbb{I}_a \\ \mathbb{I}_b \end{pmatrix} \xrightarrow[\text{operation}]{\text{Special}} \begin{pmatrix} a & + & \infty \cdot \mathbb{I}_b & + & \infty \cdot \mathbb{I}_b & + & -\infty \cdot \mathbb{I}_b \\ b & + & \infty \cdot \mathbb{I}_a & + & \infty \cdot \mathbb{I}_a & + & -\infty \cdot \mathbb{I}_a \\ \mathbb{I}_a & + & 0 & + & 0 & + & -\mathbb{I}_a \\ \mathbb{I}_b & + & 0 & + & 0 & + & -\mathbb{I}_b \end{pmatrix}$$

Note that if $\mathbb{I}_a = 1$, then $\mathbb{I}_b = 0$, so the first coordinate isn't modified and the second is changed to 0 because of the finite representation properties. In the same way, if $\mathbb{I}_a = 0$, then $\mathbb{I}_b = 1$, so the second coordinate isn't modified and the first is changed to 0.

Let us show how to instantiate the FF layers. The first two have parameters:

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & 0 & 0 & \infty \\ 0 & 0 & \infty & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

Notice that the output of these FF layer when inputted with $(x, y, \mathbb{I}_a, \mathbb{I}_b)$ will be $(\infty \cdot \mathbb{I}_b, \infty \cdot \mathbb{I}_a, 0, 0)$.

The third FF layer has parameters:

$$W_1 = id \qquad W_2 = \begin{pmatrix} 0 & 0 & 0 & -\infty \\ 0 & 0 & -\infty & 0 \\ 0 & 0 & -1 & 0 \\ 0 & 0 & 0 & -1 \end{pmatrix} \qquad b_1 = b_2 = \begin{pmatrix} 0 \\ 0 \\ 0 \\ 0 \end{pmatrix}$$

which inputted with $(x, y, \mathbb{I}_a, \mathbb{I}_b)$, it will output $(-\infty \cdot \mathbb{I}_b, -\infty \cdot \mathbb{I}_a, -\mathbb{I}_a, -\mathbb{I}_b)$

RR intenté escribir cómo quedaban las FF instanciadas pero queda medio ilegible. No sé si vale la pena

3.1.7 Repeat a token once it's printed

We show how to make a transformer repeat a token until we decide it to make it stop. This construction is needed to argue that for every transformer TF there is a transformer TF' of the same $\mathsf{CoT}[T(n), d(n)]$ class that computes the same language and consumes all of it's budget.

Let assume TF normalized and define TF'. TF' is exactly as TF except for the following modifications. Let σ be the token to repeat. We flag the vectors coming from σ with the token embedding. Since we are replicating a token that was printed, i.e., it is not part of the input, its corresponding vector is the last one. This is because we start repeating it after the first time is printed.

Since we flag the vectors coming from the token σ we can apply a constant function only to them. This function is applied in the last layer of the body of TF'. Since we assume TF was normalized, the OUTPUT matrix is just a projection. Then, the value of the constant function is what is outputted as the probability distribution (after softmax). If the constant function has value $-\infty$ in every coordinate except in the one associated to the probability of σ, where it has ∞, the distribution outputted by the distribution generator will have all the probability concentered in σ. Thus, TF' prints σ.

Notice that this only works if the token does not appear in the input word. That case can be solved using the non-uniformity of the model. If the input word is of size n, with the PE we can flag the first n vectors. Then, we can use that flag to make them 0 in the coordinate used by the TE for flagging the token to repeat. We make them 0 using a constant function as described in the section 3.1.3.

3.1.8 Force to halt after certain number of steps of CoT

With a similar strategy to the section 3.1.7, with the PE we flag when a certain position is reached. Then, if the transformer is normalized, modifying the last vector we decide the outputted distribution. Particularly, we can force it to concenter the probability in q_{accept} or q_{reject}. We change the last vector with a constant function. The flag inserted by the PE is to decide if the function has to be applied or not.

Notice that this primitive with the one described in section 3.1.7, prove that for every transformer TF in $\mathsf{CoT}[T(n), d(n)]$ there is a transformer TF' in the same class that always consumes all of its budget.

We extend the alphabet with two dummy versions of the decision tokens and construct TF' over TF. We make TF' work as TF, but we make it print the dummy versions of the decision tokens when TF would have printed the real ones[1]. Then, we force TF' to repeat the dummy decision tokens until it reaches the last position of the tape (uses all the budget). At that point, we force TF' to print the real decision token corresponding to the one it was repeating.

3.1.9 Copy one vector into another with ATTN

For the disjunction of transformers we will need to move values from the ith vector to the jth vector, in this section we show how to do it. This operation can only be done if $i < j$, since the attention mechanism only allows for a vector to attend to vectors to its left, i.e, h_k can attend to h_l if and only if $k \leq l$.

Let j be the number of the receiving vector and i the number of the copied, i.e., we copy from h_i to h_j. The operation is done in two steps.

First, we make 0 the coordinates of h_j that will receive the values. This is done with a constant function.

Second, we make h_j only attend to h_i in an attention layer. In said attention layer, we use $W_V = id$ and W_O such that it outputs the desire coordinates of h_i in the coordinates we want to copy them. Formally, W_O has only 0 in the row k if h_j is not receiving anything

[1] This is achieved giving the probability of each decision token to its dummy version.

from h_i in the kth coordinate. If h_j is receiving the coordinate l of h_i in the position k, then the kth row of W_O has 0 in every position except the lth.

For example if we want to copy the 2th coordinate of h_i to the 3th coordinate of h_j, W_O would look like 3.1. Then, the vector coming out of the ATTN layer will have 0 in every coordinate, except in the 3th, where it will be $(h_i)_2$.

$$W_O = \begin{pmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 1 & 0 \end{pmatrix} \tag{3.1}$$

3.2 Normalization

For an easier handling of transformers, we define a notion of normalized transformer. This notion will specify properties of the OUTPUT matrix and of the vector consumed by this step.

Definition 3.2.1 (Normalized transformer)**.** We will say a transformer is normalized when it meets two conditions. First, the matrix of the OUTPUT has to be a projection, i.e., it's a diagonal matrix with only ones in its diagonal. Second, the last vector after the last iteration has to only be composed of $-\infty$ in every projected coordinate except, one which has to be ∞. The non projected coordinates are free to be any value.

SF veo que más adelante se habla de la "first condition" y la "second condition". Sería bueno numerarlas o darles in nombre para que sea más fácil referirse a ellas.

The motivation for this definition is to simplify the OUTPUT layer (first constraint) and to have more control over the probability distribution generated (second constraint). The exact values of the last vectors are such that after projecting and applying softmax in the OUTPUT layer we get a deterministic distribution with 100% of the probability concentered in only one token.

SF Decí que lo que sigue es la implementación de un algoritmo para normalizar cualquier transformer.

Algorithm for normalizing

not sure about title

Let's see how to normalize any transformer. Let's take a transformer and modify it to meet the first constraint.

SF: "first constraint" se puede cambiar por el número o nombre que le hayas dado a la primera condición en la definición. Lo mismo para la segunda.

Every linear transformation $M : \mathbb{R}^d \to \mathbb{R}^{|\Sigma|}$ with $d \geq |\Sigma|$ can be decomposed in two steps. [2]

$$\begin{pmatrix} x_1 \\ \vdots \\ x_d \end{pmatrix} \xrightarrow{M'} \begin{pmatrix} (Mx)_1 \\ \vdots \\ (Mx)_{|\Sigma|} \\ v \end{pmatrix} \xrightarrow{projection} \begin{pmatrix} (Mx)_1 \\ \vdots \\ (Mx)_{|\Sigma|} \end{pmatrix}$$

where $v \in \mathbb{R}^{d-|\Sigma|}$ can have any values.

[2] We can always add more coordinates to the embedding if $d < |\Sigma|$. Since the language is fixed, it will be a constant number of coordinates, thus not changing the class of the transformer.

Let's TF be a transformer and M be the matrix of the OUTPUT layer. Using the decomposition just presented, and the strategy defined in the previous section for applying linear transformations , we can change the matrix of the OUTPUT layer to be only a projection. The linear transformation has to be applied to the last vector.

SF: acá podés ser más preciso, si ponés números a secciones

Let's now make it meet the second condition.

Once TF has been modified to meet the first condition, we can use the maximum operation described in the previous section to make all the positions of the vector 0 except the one that was the maximum before. Now, if the maximum is positive, multiplying by ∞, then subtracting 1 to every coordinate and multiplying by ∞ again gets us what we wanted.

$$\begin{pmatrix} 0 \\ \vdots \\ 0 \\ x_{max} \\ 0 \\ \vdots \\ 0 \end{pmatrix} \xrightarrow[\infty]{\text{multiplying by}} \begin{pmatrix} 0 \\ \vdots \\ 0 \\ \infty \\ 0 \\ \vdots \\ 0 \end{pmatrix} \xrightarrow{\text{substracting 1}} \begin{pmatrix} -1 \\ \vdots \\ -1 \\ \infty - 1 \\ -1 \\ \vdots \\ -1 \end{pmatrix} \xrightarrow[\infty]{\text{multiplying by}} \begin{pmatrix} -\infty \\ \vdots \\ -\infty \\ \infty \\ -\infty \\ \vdots \\ -\infty \end{pmatrix}$$

It could be the case that the maximum was negative, in that case the resulting vector won't be what we are looking for. Let's see how to make the maximum strictly positive so we can add this mechanism in the middle.

By this point our vector looks like $(0, \dots, 0, x_{max}, 0, \dots, 0)$, we are going to transform it to $(0, \dots, 0, |x_{max}|, 0, \dots, 0)$. Remember that the coordinates we are interested in are only the first $|\Sigma|$. Let's call v to the block containing these coordinates. Our algorithm will do the following:

$$\begin{pmatrix} v \\ \vdots \end{pmatrix} \to \begin{pmatrix} v \\ -v \\ \vdots \end{pmatrix} \to \begin{pmatrix} \mathrm{relu}(v) \\ \mathrm{relu}(-v) \\ \vdots \end{pmatrix} \to \begin{pmatrix} \mathrm{relu}(v) + \mathrm{relu}(-v) \\ \vdots \end{pmatrix}$$

where relu is applied position wise. This works since for all x it holds that $\mathrm{relu}(x) + \mathrm{relu}(-x) = |x|$.

The first step can be done in the same way as the first half of the linear transformations. For the second step we can take max coordinate in v and $-v$. This works because all the coordinates are 0 except one, which if it is negative will become 0 and if it is non-negative will remain the same. For adding both vectors we can use one FF with the following parameters. Notice that this works since all the coordinates of $\mathrm{relu}(-v)$ are non-negative.

$$W_1 = id^{d \times d} \qquad W_2 = \begin{pmatrix} 0^{|\Sigma| \times |\Sigma|} & id^{|\Sigma| \times |\Sigma|} & 0^{|\Sigma| \times (d - 2|\Sigma|)} \\ 0^{d - |\Sigma| \times |\Sigma|} & 0^{d - |\Sigma| \times |\Sigma|} & 0^{d - |\Sigma| \times d - 2|\Sigma|} \end{pmatrix} \qquad b_1 = b_2 = \left(0^{d \times d}\right)$$

3.3 Boolean Operations

The main objective of this chapter was to prove that the $\mathsf{CoT}[T(n), d(n)]$ classes are closed under boolean operations.

SF: section. Además, usar prsente, no pasado

Let's show that the complement of a language in $\mathsf{CoT}[T(n), d(n)]$ is in $\mathsf{CoT}[T(n), d(n)]$ and that the disjunction of two language of the same class, stays in the same class.

3.3.1 Complement of a language

Let's $\mathcal{L} \in \mathsf{CoT}[T(n), d(n)]$, then there is a family of transformers $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ of embedding size $d(n)$ that after $T(n)$ steps of CoT prints q_{accept} or q_{reject} depending on if the input word is in the language. For each of the transformers in the family we can define one that behaves exactly the same (internally and externally) but prints the flipped token.

Let's TF be a transformer, we will construct TF' that flips the decision token of TF, i.e., if $\mathsf{TF}^*(\alpha) = q_{accept}$, then $\mathsf{TF}'^*(\alpha) = q_{reject}$ and if $\mathsf{TF}^*(\alpha) = q_{reject}$, then $\mathsf{TF}'^*(\alpha) = q_{accept}$.

This can be easily done swapping the rows associated to q_{accept} and q_{reject} in the matrix of the OUTPUT layer of TF. Thus, when the largest probability is associated to q_{accept} in TF, that same number will be associated to q_{reject} in TF.

Formally, TF' will be exactly the same as TF except in the OUTPUT layer, where the matrix will be different. If M is the matrix of the OUTPUT layer of TF, i is the position of the token q_{accept} and j the one of q_{reject}, then the matrix of the OUTPUT layer of TF will be $M' = PM$ where P is the matrix that flips the rows i and j.

$$M'_{kl} = \begin{cases} 1 & \text{if } k = l,\ k \neq i \text{ and } k \neq j \\ 1 & \text{if } k = j \text{ and } l = i \\ 1 & \text{if } k = i \text{ and } l = j \\ 0 & \text{otherwise} \end{cases}$$

TF' has the same budget and embedding size that TF.

We can define the family $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ where TF'_i is the negation of TF_i. Then $\{\mathsf{TF}'_n\}_{n\in\mathbb{N}}$ computes the complement of $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$. Since for every $i \in \mathbb{N}$, TF'_i has the same budget and embedding size that TF_i, $\bar{\mathcal{L}}$ is in $\mathsf{CoT}[T(n), d(n)]$.

3.3.2 Disjunction of two languages

Let's $\mathcal{L}_1, \mathcal{L}_2 \in \mathsf{CoT}[T(n), d(n)]$, then there are is a family of transformers $\{\mathsf{TF1}_n\}_{n\in\mathbb{N}}$ (resp. $\{\mathsf{TF2}_n\}_{n\in\mathbb{N}}$) of embedding size $d_1(n)$ (resp. $d_2(n)) \in O(d(n))$ that after $T_1(n)$ (resp. $T_2(n)$) $\in T(n)$ steps of CoT that computes $\mathcal{L}_1$ (resp. $\mathcal{L}_2$).

We will construct a family $\{\mathsf{TF}_n\}_{n\in\mathbb{N}}$ that will compute $\mathcal{L}_1 \cup \mathcal{L}_2$. It will do it in $T_3(n) = T_1(n) + T_2(n) + 1 \in O(T(n))$ steps of CoT and it will have embedding size $d_3(n) \in O(d(n))$.

The idea behind the construction will be to run first one of the transformers, then run the other and finally compare both results. Let's assume that when the input is α, the tape of the transformer $\mathsf{TF1}$ ends up looking like $\alpha\sigma$q1, where $\sigma \in \Sigma^*$ and q1 is a decision symbol of $\mathsf{TF1}$. In the same way, let's call $\alpha\tau$q2 to the content of the tape of $\mathsf{TF2}$ when it halts after being input with α. The tape of the transformer computing the disjunction of $\mathsf{TF1}$ and $\mathsf{TF2}$, at the end of the computation will look like $\alpha\sigma$q1τq2q3, where σ and τ are the intermediate tokens generated by $\mathsf{TF1}$ and $\mathsf{TF2}$, q1 and q2 are their acceptance or rejection tokens and q3 is the disjunction of q1 and q2.

SF: Lo malo de usar q1 o q2 sin subíndices es que cuando los concatenás, se confunde con dos palabras.

Let's construct $\mathsf{TF3}$ from $\mathsf{TF1}$ and $\mathsf{TF2}$. Since we can't change the body of the transformer during the iterations we will require to use some tricks.

We grow the dimension of the embedding for running the body of both transformers at the same time over the same vectors but independently. In the final layers we will choose which probability distribution will go to the output layer, this will make TF3 print the token generated by TF1 or TF2. As always, we will assume that the transformers are normalized.

We will use the first half of the embedding for the transformer TF1 and the second for the transformer TF2. We will have extra coordinates for flags at the end, but we will ignore them by now.

$$\mathsf{TE3}(\sigma)_i = \begin{cases} \mathsf{TE1}(\sigma)_i & \text{if } i \leq d_1(|\alpha|) \\ \mathsf{TE2}(\sigma)_{i-d_2(|\alpha|)} & \text{if } d_1(|\alpha|) < i \leq d_2(|\alpha|) \\ 0 & \text{if } d_2(|\alpha|) < i \end{cases}$$

$$\mathsf{PE3}(n)_i = \begin{cases} \mathsf{PE1}(n)_i & \text{if } i \leq d_1(|\alpha|) \\ \mathsf{PE2}(n)_{i-d_1(|\alpha|)} & \text{if } d_1(|\alpha|) < i \leq d_2(|\alpha|) \\ \text{flags} & \text{if } d_2(|\alpha|) < i \end{cases}$$

there is a problem in this definition, PE2 is defined over $[|\alpha| + T2(|\alpha|)]$ and here we are using it over $[|\alpha| + T2(|\alpha|) + T1(|\alpha|)]$. This is fixed later on but im not sure if i want to note it right now.

The transformer TF3 will have one layer for each layer of TF1 and each of TF2. The first layers, will correspond to the body of TF1. They will be constructed ignoring the half of the embedding where the coordinates coming from TF2 are. The second half will be the other way around but with the layers of TF2 and ignoring TF1 coordinates. Then, all the matrix in the layers of the first part of the body of TF3 will look like $W3_1$ and the ones in the second layers will look like $W3_2$ where $W1$ and $W2$ are the matrix of the corresponding layers in TF1 and TF2.

SF: matrices con nombres no estándar y confusos

$$W3_1 = \begin{pmatrix} W1 & 0 \\ 0 & 0 \end{pmatrix} \qquad\qquad W3_2 = \begin{pmatrix} 0 & 0 \\ 0 & W2 \end{pmatrix}$$

It is important to notice that the values of the coordinates corresponding to one transformer does not affect the values of the other. Thus, if the tape of the transformer has content β, then at the end of the body of TF3, the last vector would look like (x, y, flags) where x is the last vector at the end of the body of TF1 when its tape content is β, and y the one at the end of TF2 when its tape content is β. Therefore, if we always choose to transform said vector into $(x, 0, \ldots, 0)$ then TF3 will produce the same tape as TF1. This works for generating the first $T1(|\alpha|)$ tokens which are σq1. Thus, we would like to apply this transformation only to the vectors generating them. Therefore, we need to flag them with the position embedding. Since $v_{|\alpha|+i}$ carries the distribution to output in the $i + 1$th step, i.e., the $|\alpha| + i + 1$th token of the tape, the vectors that we need to flag will be $v_{|\alpha|}, v_{|\alpha|+1}, \ldots, v_{|\alpha|+T1(|\alpha|)-1}$.

After running $T1(|\alpha|)$ steps of this construction, the tape will look like $\alpha\sigma$q1. We want to now generate τ. Just changing what we choose before the OUTPUT layer does not work since what the construction will produce would be TF2($\alpha\sigma$q1), and we are looking for TF2(α).

Let's see how to run TF2 in this construction ignoring the tokens produced by TF1. If we are able to make those tokens invisible to TF2, we are done. We need for TF2 to not attend to them (this can be done using the primitive presented in the previous section) and for its position encoding to ignore them, therefore we need to "shift" it. Let's see first how to fix the position encoding of TF3.

$$\mathsf{PE3}(n)_i = \begin{cases} \mathsf{PE1}(n)_i & \text{if } i \leq d_1(|\alpha|) \\ \text{shifted}\mathsf{PE2}(n)_{i-d_1(|\alpha|)} & \text{if } d_1(|\alpha|) < i \leq d_2(|\alpha|) \\ \text{flags} & \text{if } d_2(|\alpha|) < i \end{cases}$$

$$\text{shifted}\mathsf{PE2}(n) = \begin{cases} \mathsf{PE2}(n) & \text{if } n \leq |\alpha| \\ 0 & \text{if } |\alpha| < n < T1(|\alpha|) \\ \mathsf{PE2}(n - T1(|\alpha|)) & \text{if } T1(|\alpha|) < n \end{cases}$$

The first case is for the positions occupied by the input, we don't want to change them. The second case is for the positions occupied by the tokens generated by TF1, we don't care about them. The third case is for the tokens that TF2 will generate, we want to make it believe that they are in their natural positions, i.e., shifted $T1(|\alpha|)$ positions left (the positions occupied by the generated for TF1). We will also need to change the flag to the vectors after the number $\alpha + T1(|\alpha|)$ to choose the distribution generated by TF2 instead of TF1.

With the construction like this we can almost produce the tape of TF1 followed by the tape of TF2 except for one token. We need to do something especial for the first token generated by TF2. The distribution corresponding to it, it's in the vector $v_{|\alpha|}$ but the last vector of the first run of TF2 is $v_{|\alpha|+T1(|\alpha|)}$, the one coming from the last token of TF1. Therefore, before choosing the distribution, we need to copy the vector. This can be done with the primitives of the previous section.

With this last fix, we are able to generate the tape of TF1 followed by the tape of TF2. We are only missing how to compute q3. This can be easily done making $v_{|\alpha|+T1(|\alpha|)+T2(\alpha)}$ only attend to itself and to $v_{|\alpha|+T1(|\alpha|)}$, i.e., to the vectors coming from q1 and q2.

We will modify the TE3 function to add a flag in the last coordinate (let's call it f_c) for the decision tokens of TF1 and TF2.

$$\mathsf{TE3}(\sigma)_{f_c} = \begin{cases} 1 & \text{if } \sigma = q_{accept} \vee \sigma = q_{accept} \\ 0 & \text{otherwise} \end{cases}$$

Then if we add $v_{|\alpha|+T1(|\alpha|)}$ to $v_{|\alpha|+T1(|\alpha|)+T2(\alpha)}$ we will have 1 or 2 in the coordinate f_c iff TF1 or TF2 accepted α. Now we can make all of $v_{|\alpha|+T1(|\alpha|)+T2(\alpha)}$ 0, except the f_c coordinate. Before going to the output layer we need to move the value of f_c to the coordinate that will project to the probability of q_{accept} and add some number larger than 0 and smaller than 1 to the coordinate that will define the probability of q_{reject}.

Therefore, the token with the maximum probability will be q_{accept} iff TF1 or TF2 accepted α, and it will be q_{reject} in the other case. Then, TF3 computes the union of the languages of TF1 and TF2. The number of steps of CoT that it needs is the same as TF1 plus TF2 plus 1 extra for taking the disjunction, thus in $O(\dot{})$ notation it takes the same as the maximum between TF1 and TF2. The embedding dimension of TF3 is the same as the dimension of TF1 plus TF2 plus some constant number of flags, thus in $O(\dot{})$ notation it's the same as the maximum between TF1 and TF2.

Therefore, the language computed by the family of transformers $\{\mathsf{TF3}_n\}_{n\in\mathbb{N}}$ is in $\mathsf{CoT}[T(n), d(n)]$. Thus, $\mathcal{L}1 \cup \mathcal{L}2 \in \mathsf{CoT}[T(n), d(n)]$.

4. MAIN RESULTS

5. CONCLUSIONS